

Python

3.12

Django 5

Supabase

Clerk Auth

Vercel

Upstash

Redis

SocialHub Pro

Plataforma de Gestão de Redes Sociais

Guia Completo de Desenvolvimento — Stack 100% Gratuita

\$0/mês

custo infra base

20+

models do banco

8

fases do guia

200+

linhas de código exemplo

Serviço	Função	Plano	Custo/mê s
GitHub	Versionamento de código	Free	R\$ 0
Claude AI	Assistente de desenvolvimento	Pro \$20	R\$ ~110
Namecheap	Domínio (.com)	Registrar	~R\$ 7/ano
Cloudflare	DNS, CDN e HTTPS	Free	R\$ 0
Vercel	Deploy do app (serverless)	Hobby	R\$ 0
Supabase	PostgreSQL + Storage	Free	R\$ 0
Clerk	Autenticação completa	Free	R\$ 0
Upstash	Redis (Celery + cache)	Free	R\$ 0
Pinecone	Vector DB (busca semântica IA)	Free	R\$ 0
Resend	E-mails transacionais	Free	R\$ 0
Stripe	Pagamentos (planos pagos)	Pay-as-go	2.9%/tx
PostHog	Analytics de produto	Free	R\$ 0
Sentry	Rastreamento de erros	Free	R\$ 0
Railway	Celery Workers	\$5 crédito	R\$ 0*

* Railway dá \$5/mês de crédito gratuito — suficiente para workers leves.

●	Parte 1	Roteiro & Estrutura de Pastas
●	Parte 2	Stack Gratuita — Setup Completo (Dev)
●	Parte 3	Modelagem do Banco de Dados — Todos os Models
●	Parte 4	Sistema de Usuários Completo
●	Parte 5	APIs Sociais, Agendamento e IA
●	Parte 6	Inbox, Auto-resposta e Live Streaming
●	Parte 7	Produção — Deploy, CI/CD, HTTPS e Monitoramento
●	Apêndice	Referência rápida e comandos do dia a dia

PARTE 1 — ROTEIRO & ESTRUTURA DE PASTAS

Antes de escrever qualquer código, é fundamental entender o plano completo e como os arquivos estão organizados. Uma estrutura bem definida desde o início evita retrabalho e facilita trabalho em equipe.

Roadmap — 8 Fases do Projeto

Fase	Nome	O que você constrói	Semanas
1	Fundação	Django, banco Supabase, Clerk, Redis, CI/CD básico	1–2
2	APIs Sociais	OAuth2, conectar Instagram/Twitter/LinkedIn/TikTok/YouTube	3–5
3	Agendamento + IA	Celery, posts automáticos, GPT-4o, hashtags trends	6–8
4	Inbox & Auto-resposta	DMs unificados, regras de resposta, moderação	9–10
5	Live & Reuniões	WebRTC/Agora.io, streaming, salas de vídeo	11–12
6	Analytics	Métricas, gráficos Chart.js, relatórios, PostHog	13–14
7	Segurança & Perf.	Rate limiting, 2FA, cache, otimização de queries	15–16
8	Produção	Docker (workers), CI/CD GitHub Actions, Sentry, HTTPS	17–18

Estrutura Completa de Pastas do Projeto

■ **Convenção Django**

Cada 'app' é uma funcionalidade independente. Separar assim permite testar, reutilizar e manter cada parte sem afetar as outras. A pasta 'config/' guarda as configurações globais do projeto.

■ Estrutura completa de pastas

```
socialhub/ ← raiz do projeto
■
■■■ config/ ← configurações globais do Django
■ ■■■ __init__.py ← importa o Celery para o Django
■ ■■■ settings/ ← settings por ambiente
■ ■ ■■■ __init__.py
■ ■ ■■■ base.py ← configurações comuns
■ ■ ■■■ development.py ← dev: debug=True, logs detalhados
■ ■ ■■■ production.py ← prod: HTTPS, Sentry, sem debug
■ ■■■ urls.py ← rotas principais
■ ■■■ wsgi.py ← para Gunicorn / Vercel
■ ■■■ asgi.py ← para Daphne / WebSocket
■ ■■■ celery.py ← configuração do Celery
■
■■■ apps/ ← todos os módulos da aplicação
■ ■■■ accounts/ ← usuários, perfis, planos, equipes
■ ■ ■■■ migrations/ ← arquivos de migration do banco
■ ■ ■■■ tests/ ← testes do app
■ ■ ■ ■■■ test_models.py
■ ■ ■ ■■■ test_views.py
■ ■ ■■■ api/ ← endpoints REST (DRF)
■ ■ ■ ■■■ serializers.py
```

```
■ ■ ■ ■■ views.py
■ ■ ■ ■■ urls.py
■ ■ ■■ templates/accounts/ ← HTMLs do app
■ ■ ■ ■■ login.html
■ ■ ■ ■■ onboarding.html
■ ■ ■ ■■ profile.html
■ ■ ■ ■■ invite.html
■ ■ ■■ admin.py
■ ■ ■■ apps.py
■ ■ ■■ clerk_auth.py ← middleware JWT do Clerk
■ ■ ■■ forms.py
■ ■ ■■ models.py ← UserProfile, Plan, Team
■ ■ ■■ permissions.py ← RBAC (roles e permissões)
■ ■ ■■ signals.py ← criar perfil ao criar usuário
■ ■ ■■ urls.py
■ ■ ■■ views.py
■ ■
■ ■■ social/ ← integração com APIs das redes
■ ■ ■■ migrations/
■ ■ ■■ services/ ← lógica de cada rede social
■ ■ ■ ■■ __init__.py
■ ■ ■ ■■ base.py ← classe base (interface comum)
■ ■ ■ ■■ instagram.py
■ ■ ■ ■■ twitter.py
■ ■ ■ ■■ linkedin.py
■ ■ ■ ■■ tiktok.py
■ ■ ■ ■■ youtube.py
■ ■ ■■ webhooks/ ← receber eventos das plataformas
■ ■ ■ ■■ instagram.py
■ ■ ■ ■■ twitter.py
■ ■ ■■ admin.py
■ ■ ■■ models.py ← SocialAccount
■ ■ ■■ urls.py
■ ■ ■■ views.py
■ ■
■ ■■ scheduler/ ← agendamento de posts
■ ■ ■■ migrations/
■ ■ ■■ tasks.py ← tarefas Celery (publicar, retry)
■ ■ ■■ admin.py
■ ■ ■■ models.py ← ScheduledPost, PostMedia, Result
■ ■ ■■ urls.py
■ ■ ■■ views.py
■ ■
■ ■■ inbox/ ← mensagens e auto-resposta
■ ■ ■■ migrations/
■ ■ ■■ consumers.py ← WebSocket (django-channels)
■ ■ ■■ routing.py ← rotas WebSocket
■ ■ ■■ tasks.py ← processar mensagens recebidas
■ ■ ■■ admin.py
■ ■ ■■ models.py ← InboxMessage, AutoReplyRule
■ ■ ■■ urls.py
■ ■ ■■ views.py
■ ■
■ ■■ analytics/ ← métricas e relatórios
■ ■ ■■ migrations/
■ ■ ■■ tasks.py ← coletar métricas periodicamente
```

```
■ ■ ■■ posthog_client.py ← wrapper do PostHog
■ ■ ■■ admin.py
■ ■ ■■ models.py ← AccountSnapshot, PostMetrics
■ ■ ■■ urls.py
■ ■ ■■ views.py
■ ■
■ ■■■ live/ ← streaming e reuniões
■ ■ ■■ migrations/
■ ■ ■■ admin.py
■ ■ ■■ models.py ← LiveRoom, LiveParticipant
■ ■ ■■ services.py ← Agora.io token generator
■ ■ ■■ urls.py
■ ■ ■■ views.py
■ ■
■ ■■■ ai_engine/ ← IA, geração de conteúdo
■ ■ ■■ services.py ← GPT-4o, hashtags, sugestões
■ ■ ■■ tasks.py
■ ■ ■■ models.py ← AIGenerationLog, HashtagCache
■ ■
■ ■■■ payments/ ← Stripe e assinaturas
■ ■■ models.py ← Subscription, Invoice
■ ■■ urls.py
■ ■■ views.py ← checkout session, webhook
■
■■■ templates/ ← templates HTML globais
■ ■■ base.html ← layout principal (extends)
■ ■■ dashboard.html
■ ■■ components/ ← componentes reutilizáveis
■ ■ ■■ navbar.html
■ ■ ■■ sidebar.html
■ ■ ■■ post_card.html
■ ■■ emails/ ← templates de e-mail HTML
■ ■■ invite.html
■ ■■ welcome.html
■
■■■ static/ ← arquivos estáticos (dev)
■ ■■ css/
■ ■ ■■ tailwind.css
■ ■■ js/
■ ■ ■■ alpine.min.js
■ ■ ■■ htmx.min.js
■ ■■ img/
■
■■■ staticfiles/ ← coletados por collectstatic (prod)
■
■■■ media/ ← uploads de desenvolvimento
■ (em produção vai para Supabase Storage)
■
■■■ .github/
■ ■■ workflows/
■ ■■ test.yml ← roda testes em todo pull request
■ ■■ deploy.yml ← deploy automático ao merge no main
■
■■■ docker/ ← Docker só para workers em produção
■ ■■ Dockerfile.worker ← imagem do Celery worker
■ ■■ docker-compose.prod.yml ← workers + beat + flower
```

-
- ■ ■ scripts/ ← scripts de manutenção
- ■ ■ ■ create_plans.py ← popula planos no banco
- ■ ■ ■ seed_data.py ← dados de teste
-
- ■ ■ .env ← variáveis de ambiente (não commitar!)
- ■ ■ .env.example ← modelo do .env para o time
- ■ ■ .gitignore
- ■ ■ manage.py
- ■ ■ pyproject.toml ← dependências (Poetry)
- ■ ■ requirements.txt ← gerado para Vercel
- ■ ■ vercel.json ← config deploy Vercel
- ■ ■ pytest.ini ← config de testes
- ■ ■ README.md

■ Settings separados por ambiente

■ config/settings/base.py

```
# config/settings/base.py – configurações COMUNS (dev e prod)

from pathlib import Path
import environ

env = environ.Env(DEBUG=(bool, False))
BASE_DIR = Path(__file__).resolve().parent.parent.parent
environ.Env.read_env(BASE_DIR / '.env')

SECRET_KEY = env('SECRET_KEY')

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Terceiros
    'rest_framework',
    'corsheaders',
    'channels',
    'django_celery_beat',
    'django_celery_results',
    # Apps do projeto
    'apps.accounts',
    'apps.social',
    'apps.scheduler',
    'apps.inbox',
    'apps.analytics',
    'apps.live',
    'apps.ai_engine',
    'apps.payments',
]

ROOT_URLCONF = 'config.urls'
WSGI_APPLICATION = 'config.wsgi.application'
ASGI_APPLICATION = 'config.asgi.application'
AUTH_USER_MODEL = 'auth.User'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
LANGUAGE_CODE = 'pt-br'
TIME_ZONE = 'America/Sao_Paulo'
```

```
USE_I18N = USE_TZ = True

STATIC_URL = '/static/'

STATIC_ROOT = BASE_DIR / 'staticfiles'

MEDIA_URL = '/media/'

# Celery
CELERY_BROKER_URL = env('REDIS_URL')
CELERY_RESULT_BACKEND = 'django-db'
CELERY_TIMEZONE = 'America/Sao_Paulo'

CELERY_BEAT_SCHEDULER = 'django_celery_beat.schedulers:DatabaseScheduler'
```

- `config/settings/development.py`

```
# config/settings/development.py
from .base import *

DEBUG = True

ALLOWED_HOSTS = ['localhost', '127.0.0.1']

# Banco de dados: Supabase (mesmo em dev – use o mesmo projeto)
import dj_database_url

DATABASES = {'default': dj_database_url.config(default=env('DATABASE_URL'))}

# E-mail no terminal (não envia de verdade em dev)
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'

# Redis local (docker compose up -d)

# OU Upstash direto – qualquer um funciona em dev

INSTALLED_APPS += ['debug_toolbar']

INTERNAL_IPS = ['127.0.0.1']
```

```
■ config/settings/production.py
```

```
# config/settings/production.py
from .base import *

DEBUG = False

ALLOWED_HOSTS = [env('ALLOWED_HOST'), 'www.' + env('ALLOWED_HOST')]

import dj_database_url

DATABASES = {
    'default': dj_database_url.config(
        default=env('DATABASE_URL'),
        conn_max_age=600,
        conn_health_checks=True,
    )
}

# HTTPS obrigatório
SECURE_HSTS_SECONDS = 31536000 # 1 ano
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
X_FRAME_OPTIONS = 'DENY'
SECURE_CONTENT_TYPE_NOSNIFF = True

# Arquivos estáticos via WhiteNoise
MIDDLEWARE.insert(1, 'whitenoise.middleware.WhiteNoiseMiddleware')
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'

# Storage de mídia no Supabase S3
DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'
AWS_STORAGE_BUCKET_NAME = env('SUPABASE_BUCKET_NAME')
AWS_S3_ENDPOINT_URL = env('SUPABASE_S3_URL')
AWS_ACCESS_KEY_ID = env('SUPABASE_ACCESS_KEY')
AWS_SECRET_ACCESS_KEY = env('SUPABASE_SECRET_KEY')
```

```
# Sentry – monitoramento de erros
import sentry_sdk
from sentry_sdk.integrations.django import DjangoIntegration
from sentry_sdk.integrations.celery import CeleryIntegration
sentry_sdk.init(
    dsn=env('SENTRY_DSN'),
    integrations=[DjangoIntegration(), CeleryIntegration()],
    traces_sample_rate=0.1,
    send_default_pii=False,
    environment='production',
)

# E-mail via Resend

EMAIL_BACKEND = 'apps.accounts.email_backend.ResendEmailBackend'
```

■ manage.py e DJANGO_ENV

```
# manage.py – apontar para o settings correto
#!/usr/bin/env python
import os, sys

def main():
    # Em dev usa development, em prod usa production
    env = os.environ.get('DJANGO_ENV', 'development')
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', f'config.settings.{env}')
    from django.core.management import execute_from_command_line
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()

# .env – variável que controla o ambiente:
DJANGO_ENV=development # em dev
# DJANGO_ENV=production # no Vercel e Railway
```

PARTE 2 — STACK GRATUITA: SETUP PASSO A PASSO

Configure cada serviço na ordem abaixo antes de escrever código do projeto. Todos têm plano gratuito permanente — não é trial.

#1

AMBIENTE LOCAL — Python, Poetry, Git

Color(.247059,.72549,.313725,1)

■ Setup inicial completo

```
# 1. Instalar pyenv (gerenciador de versão Python)
curl https://pyenv.run | bash
# Adicione ao ~/.bashrc ou ~/.zshrc:
export PYENV_ROOT="$HOME/.pyenv"
export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"

# 2. Instalar Python 3.12
pyenv install 3.12.3 && pyenv global 3.12.3
python --version # Python 3.12.3

# 3. Instalar Poetry
curl -sSL https://install.python-poetry.org | python3 -
poetry config virtualenvs.in-project true # .venv dentro do projeto

# 4. Criar o projeto
mkdir socialhub && cd socialhub
git init && git branch -M main

# 5. Criar pyproject.toml com Poetry
poetry init --name socialhub --python '>=3.12,<3.13' --no-interaction

# 6. Instalar Django e dependências principais
poetry add django==5.0.6 djangorestframework django-enviro
poetry add dj-database-url psycopg2-binary
poetry add celery[redis] django-celery-beat django-celery-results
poetry add channels[daphne] channels-redis
poetry add django-storages boto3
poetry add django-cors-headers whitenoise
poetry add PyJWT cryptography requests
poetry add openai stripe resend posthog
poetry add sentry-sdk[django]
poetry add Pillow

# 7. Dependências de desenvolvimento
poetry add --group dev pytest pytest-django factory-boy ruff ipython

# 8. Criar estrutura do projeto
poetry run django-admin startproject config .
mkdir -p apps templates static media scripts .github/workflows docker

# 9. Criar todos os apps
cd apps
for app in accounts social scheduler inbox analytics live ai_engine payments; do
poetry run python ../manage.py startapp $app
done
cd ..

# 10. Criar .gitignore
echo ".env .venv/ __pycache__/ *.pyc media/ staticfiles/ db.sqlite3 .DS_Store" > .gitignore
```

```
# 11. Primeiro commit

git add . && git commit -m "chore: estrutura inicial do projeto"
```

#2

SUPABASE — PostgreSQL + Storage (gratuito)

Color(.176471,.831373,.74902,1)

■ Supabase — configuração

```
# 1. Acesse: supabase.com → Sign Up com GitHub
# 2. New Project:
# Name: socialhub-prod
# Password: [senha forte – anote!]
# Region: South America (São Paulo)
#
# 3. Após criar (leva ~2 min), vá em:
# Settings → Database → Connection string → URI
# Copie a URL (começa com postgres://)
#
# 4. Storage: vá em Storage → New bucket
# Name: socialhub-media
# Public bucket: SIM (para fotos de perfil e mídias públicas)
# Vá em Settings → S3 Connection → copie as credenciais
# .env – adicionar:
DATABASE_URL=postgres://postgres:SENHA@db.SEU_ID.supabase.co:5432/postgres
SUPABASE_S3_URL=https://SEU_ID.supabase.co/storage/v1/s3
SUPABASE_BUCKET_NAME=socialhub-media
SUPABASE_ACCESS_KEY=xxxxxx
SUPABASE_SECRET_KEY=xxxxxx
# Testar conexão:
poetry run python manage.py migrate
# Se aparecer 'OK' nas migrações → banco conectado!
```

#3

CLERK — Autenticação completa (gratuito)

Color(.737255,.54902,1,1)

■ Clerk + ClerkAuthMiddleware

```
# 1. clerk.com → Create Account (grátis com GitHub)
# 2. Create Application:
# Name: SocialHub Pro
# Sign-in options: Email, Google, GitHub
#
# 3. Habilite provedores sociais:
# Configure → Social Connections → ative Google, GitHub, Facebook
#
# 4. API Keys → copie:
# Publishable Key: pk_test_xxx (para o frontend)
# Secret Key: sk_test_xxx (para o backend)
# JWKS URL: https://SEU_APP.clerk.accounts.dev/.well-known/jwks.json
# .env – adicionar:
CLERK_SECRET_KEY=sk_test_XXXXXXXXXXXXXXXXXX
CLERK_JWKS_URL=https://SEUAPP.clerk.accounts.dev/.well-known/jwks.json
# apps/accounts/clerk_auth.py
```

```
import jwt, requests, json
from django.conf import settings
from django.contrib.auth.models import User

def _get_jwks():
    r = requests.get(settings.CLERK_JWKS_URL, timeout=10)
    return r.json()

def verify_clerk_token(token: str) -> dict | None:
    try:
        jwks = _get_jwks()
        return jwt.decode(token, jwks, algorithms=['RS256'],
            options={'verify_aud': False})
    except Exception:
        return None

def get_or_create_user(payload: dict) -> User:
    clerk_id = payload['sub']
    email = payload.get('email', '')
    first_name = payload.get('first_name', '')
    last_name = payload.get('last_name', '')
    try:
        from apps.accounts.models import UserProfile
        return UserProfile.objects.get(clerk_user_id=clerk_id).user
    except UserProfile.DoesNotExist:
        pass
    user, _ = User.objects.get_or_create(
        email=email, defaults={'username': email,
            'first_name': first_name,
            'last_name': last_name})
    user.profile.clerk_user_id = clerk_id
    user.profile.email_verified = True
    user.profile.save()
    return user

class ClerkAuthMiddleware:
    def __init__(self, get_response): self.get_response = get_response
    def __call__(self, request):
        auth = request.headers.get('Authorization', '')
        if auth.startswith('Bearer '):
            payload = verify_clerk_token(auth[7:])
            if payload: request.user = get_or_create_user(payload)
        return self.get_response(request)
```

#4

UPSTASH — Redis serverless (gratuito)

Color(.890196,.701961,.254902,1)

■ Upstash

```
# 1. upstash.com → Sign Up (grátis com GitHub)
# 2. Create Database:
# Name: socialhub-redis
# Region: South America
# Type: Regional
#
# 3. Details → Connect → copie a Redis URL:
# redis://:SUA_SENHA@global-xxx.upstash.io:6379
# .env:
```

```
REDIS_URL=redis://:SENHA@global-xxx.upstash.io:6379

# Testar conexão (no shell Django):
poetry run python manage.py shell
from django.core.cache import cache
cache.set('teste', 'funcionou', 30)

print(cache.get('teste')) # → 'funcionou'
```

#5

RESEND — E-mails (3.000/mês gratuitos)

Color(968627,.470588,.729412,1)

■ Resend

```
# 1. resend.com → Sign Up → API Keys → Create
# 2. Domains → Add Domain → seguir instruções DNS no Cloudflare

# .env:
RESEND_API_KEY=re_XXXXXXXXXXXXXXXXXXXX
DEFAULT_FROM_EMAIL=noreply@seudominio.com

# apps/accounts/email_backend.py
import resend
from django.conf import settings
from django.core.mail.backends.base import BaseEmailBackend

class ResendEmailBackend(BaseEmailBackend):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        resend.api_key = settings.RESEND_API_KEY

    def send_messages(self, email_messages):
        sent = 0
        for msg in email_messages:
            try:
                body = getattr(msg, 'alternatives', None)
                html = body[0][0] if body else f'{msg.body}'
                resend.Emails.send({'from': settings.DEFAULT_FROM_EMAIL,
                                    'to': msg.recipients(),
                                    'subject': msg.subject,
                                    'html': html})
                sent += 1
            except Exception:
                if not self.fail_silently: raise
        return sent
```

#6

VERCEL — Deploy do Django (gratuito)

Color(.345098,.65098,1,1)

■ Vercel — deploy

```
# vercel.json — na raiz do projeto
{
  "version": 2,
  "builds": [{"src": "config/wsgi.py", "use": "@vercel/python"}],
  "routes": [{"src": "/(.*)", "dest": "config/wsgi.py"}],
  "env": {"DJANGO_ENV": "production"}
}

# config/wsgi.py
```

```
import os
from django.core.wsgi import get_wsgi_application
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings.production')
application = get_wsgi_application()
app = application # ← Vercel precisa desta variável

# Gerar requirements.txt (Vercel usa isso, não pyproject.toml)
poetry export -f requirements.txt --output requirements.txt --without-hashes

# Deploy:
npm install -g vercel
vercel login
vercel # configura o projeto (só na 1ª vez)
vercel --prod # deploy de produção

# Variáveis de ambiente (IMPORTANTE):
# Vercel → seu projeto → Settings → Environment Variables
# Adicione todas as variáveis do .env com scope 'Production'

# Após configurar o domínio:
# Vercel → Domains → Add → seudominio.com

# O Vercel mostra os registros DNS para adicionar no Cloudflare
```

#7

RAILWAY — Celery Workers (gratuito com crédito \$5)

Color(,247059,,72549,,313725,1)

■ ■ Por que Railway?

O Vercel é serverless — funções que nascem e morrem por requisição. O Celery precisa de um processo que fique rodando 24/7 aguardando tarefas. O Railway.app dá \$5/mês de crédito gratuito — suficiente para 2 workers leves.

■ Railway — workers Celery

```
# docker/Dockerfile.worker
FROM python:3.12-slim
RUN apt-get update && apt-get install -y gcc libpq-dev && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
ENV DJANGO_ENV=production
# CMD é definido no railway.json por serviço
# railway.json — na raiz
{
  "services": {
    "worker": {
      "startCommand": "celery -A config worker --loglevel=info --concurrency=2"
    },
    "beat": {
      "startCommand": "celery -A config beat --loglevel=info"
    }
  }
}

# railway.app → New Project → Deploy from GitHub
# Selecione o repositório → adicione as env vars
# Crie 2 serviços: worker e beat (com os comandos acima)
```

PARTE 3 — MODELAGEM DO BANCO DE DADOS

Todos os modelos Django do projeto, com campos detalhados, relacionamentos e código completo. Crie os apps e models na ordem listada para evitar erros de ForeignKey para models ainda não existentes.

Ordem de criação (evita erros de FK)

Ordem	App	Model	Depende de
1°	accounts	Plan	—
2°	accounts	UserProfile	User (Django), Plan
3°	accounts	Team	User, Plan
4°	accounts	TeamMembership	Team, User
5°	accounts	Notification	User
6°	accounts	AuditLog	User
7°	social	SocialAccount	User, Team
8°	scheduler	ScheduledPost	User, Team, SocialAccount
9°	scheduler	PostMedia	ScheduledPost
10°	scheduler	PublicationResult	ScheduledPost, SocialAccount
11°	inbox	InboxMessage	User, SocialAccount
12°	inbox	InboxReply	InboxMessage, User
13°	inbox	AutoReplyRule	User
14°	analytics	AccountSnapshot	SocialAccount
15°	analytics	PostMetrics	SocialAccount
16°	live	LiveRoom	User
17°	live	LiveParticipant	LiveRoom, User
18°	payments	Subscription	User, Plan
19°	ai_engine	AIGenerationLog	User
20°	ai_engine	HashtagCache	—

App: accounts

Model: Plan + UserProfile

Plan	UserProfile
id PK name CharField Free / Pro / Business slug SlugField ♦ free / pro / business stripe_price_id CharField price_xxx do Stripe price_monthly Decimal R\$0 / R\$49.90 / R\$149 price_yearly Decimal com desconto anual max_social_accounts Integer 1 / 5 / 20 max_scheduled_posts Integer 10 / 500 / ilimitado max_team_members Integer 1 / 3 / 20 storage_gb Integer GB de armazenamento has_ai_features Boolean geração por IA has_analytics Boolean métricas detalhadas has_live_streaming Boolean live e reuniões has_auto_reply Boolean respostas automáticas has_api_access Boolean acesso à API externa is_active Boolean disponível para compra?	id PK user 1:1 → User ♦ User do Django clerk_user_id CharField ♦ ID no Clerk (único) bio TextField avatar ImageField Supabase Storage phone CharField website URLField timezone CharField America/Sao_Paulo language CharField pt-br / en plan FK → Plan plano ativo plan_expires_at DateTime stripe_customer_id CharField ID no Stripe email_verified Boolean verificado via Clerk two_factor_enabled Boolean onboarding_done Boolean tutorial concluído? created_at DateTime auto updated_at DateTime auto

■ apps/accounts/models.py — Plan + UserProfile

```
# apps/accounts/models.py
from django.db import models
from django.contrib.auth.models import User
from django.db.models.signals import post_save
from django.dispatch import receiver

class Plan(models.Model):
    name = models.CharField(max_length=50)
    slug = models.SlugField(unique=True)
    stripe_price_id = models.CharField(max_length=100, blank=True)
    price_monthly = models.DecimalField(max_digits=8, decimal_places=2, default=0)
    price_yearly = models.DecimalField(max_digits=8, decimal_places=2, default=0)
    max_social_accounts = models.IntegerField(default=1)
    max_scheduled_posts = models.IntegerField(default=10)
    max_team_members = models.IntegerField(default=1)
    storage_gb = models.IntegerField(default=1)
    has_ai_features = models.BooleanField(default=False)
    has_analytics = models.BooleanField(default=False)
    has_live_streaming = models.BooleanField(default=False)
    has_auto_reply = models.BooleanField(default=False)
    has_api_access = models.BooleanField(default=False)
    is_active = models.BooleanField(default=True)

    class Meta:
        ordering = ['price_monthly']
    def __str__(self):
        return f'{self.name} (R${self.price_monthly}/mês)'

class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
                                related_name='profile')
    clerk_user_id = models.CharField(max_length=100, unique=True, blank=True)
    bio = models.TextField(blank=True)
    avatar = models.ImageField(upload_to='avatars/', blank=True, null=True)
    phone = models.CharField(max_length=20, blank=True)
    website = models.URLField(blank=True)
```

```
timezone = models.CharField(max_length=50, default='America/Sao_Paulo')
language = models.CharField(max_length=10, default='pt-br')
plan = models.ForeignKey(Plan, on_delete=models.SET_NULL,
null=True, blank=True, related_name='subscribers')
plan_expires_at = models.DateTimeField(null=True, blank=True)
stripe_customer_id = models.CharField(max_length=100, blank=True)
email_verified = models.BooleanField(default=False)
two_factor_enabled = models.BooleanField(default=False)
onboarding_done = models.BooleanField(default=False)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)

@property
def full_name(self):
    return f'{self.user.first_name} {self.user.last_name}'.strip() or self.user.username

@property
def is_plan_active(self):
    from django.utils import timezone
    if not self.plan: return False
    if not self.plan_expires_at: return True # plano permanente
    return self.plan_expires_at > timezone.now()

@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        free_plan = Plan.objects.filter(slug='free').first()
        UserProfile.objects.create(user=instance, plan=free_plan)
```

■ Model: Team + TeamMembership

Team	TeamMembership
id PK name CharField nome da agência/equipe slug SlugField na URL description TextField avatar ImageField owner FK → User dono/criador plan FK → Plan plano da equipe plan_expires_at DateTime members M2M → User via TeamMembership is_active Boolean created_at DateTime auto	id PK team FK → Team user FK → User role CharField owner/admin/editor/viewer invited_by FK → User quem convidou invite_accepted Boolean aceitou o convite? invite_token UUID token único do link joined_at DateTime quando aceitou created_at DateTime auto

■ App: social

■ Model: SocialAccount

SocialAccount
id PK user FK → User dono da conta team FK → Team equipe (opcional)

```
platform CharField instagram/twitter/youtube...

platform_user_id CharField ⚡ ID único na plataforma

username CharField @handle

display_name CharField nome exibido

avatar_url URLField foto de perfil

bio TextField

followers_count Integer atualizado a cada 6h

following_count Integer

posts_count Integer

access_token TextField OAuth2 – NUNCA expor

refresh_token TextField

token_expires_at DateTime

scopes TextField permissões concedidas

is_active Boolean

connected_at DateTime auto

last_synced_at DateTime última sinc. de métricas
```

■ App: scheduler

■ Models: ScheduledPost + PostMedia + PublicationResult

ScheduledPost	PostMedia
id PK user FK → User ⚡ team FK → Team accounts M2M → SocialAccount redes de destino content TextField texto principal content_per_platform JSONField {'twitter': '...'} hashtags TextField #tag1 #tag2 first_comment TextField 1º comentário (Instagram) scheduled_at DateTime ⚡ quando publicar timezone CharField status CharField draft/scheduled/published/failed published_at DateTime quando foi publicado retry_count Integer máx 3 tentativas error_message TextField em caso de falha ai_generated Boolean ai_prompt TextField prompt usado created_at DateTime auto updated_at DateTime auto	id PK post FK → ScheduledPost ⚡ file FileField Supabase Storage media_type CharField image/video/gif original_filename CharField file_size Integer bytes width Integer px height Integer px duration_seconds Float se for vídeo thumbnail ImageField alt_text CharField acessibilidade order Integer carrossel created_at DateTime auto

PublicationResult
id PK post FK → ScheduledPost ⚡ account FK → SocialAccount ⚡

```
platform_post_id CharField ID do post publicado

platform_post_url URLField link público

status CharField success/failed/skipped

error_code CharField código HTTP ou da API

error_message TextField

published_at DateTime auto
```

■ Apps: inbox, analytics, live

■ Models: InboxMessage + AutoReplyRule

InboxMessage	AutoReplyRule
<code>id</code> PK <code>user</code> FK → User ♦ dono do inbox <code>account</code> FK → SocialAccount♦ conta que recebeu <code>platform_message_id</code> CharField ♦ ID único na plataforma <code>message_type</code> CharField dm/comment/mention <code>sender_platform_id</code> CharField ID do remetente <code>sender_username</code> CharField @username <code>sender_display_name</code> CharField <code>sender_avatar_url</code> URLField <code>content</code> TextField texto da mensagem <code>media_url</code> URLField imagem/vídeo <code>is_read</code> Boolean <code>is_replied</code> Boolean <code>auto_replied</code> Boolean respondido por bot? <code>is_hidden</code> Boolean moderado <code>sentiment</code> CharField positive/neutral/negative <code>language</code> CharField idioma detectado <code>received_at</code> DateTime ♦ horário original <code>created_at</code> DateTime auto	<code>id</code> PK <code>user</code> FK → User ♦ <code>name</code> CharField ex: Responder preço <code>applies_to</code> CharField dm/comment/both <code>trigger_type</code> CharField keyword/all/sentiment <code>trigger_keywords</code> TextField palavra1, palavra2 <code>reply_mode</code> CharField fixed/ai_generated <code>fixed_reply</code> TextField <code>ai_instructions</code> TextField contexto para a IA <code>ai_tone</code> CharField formal/casual/friendly <code>max_per_hour</code> Integer limite respostas/hora <code>max_per_day</code> Integer <code>active_hours_start</code> Integer hora início (0-23) <code>active_hours_end</code> Integer hora fim (0-23) <code>is_active</code> Boolean <code>reply_count</code> Integer total enviadas <code>created_at</code> DateTime auto

■ Models: AccountSnapshot + LiveRoom

AccountSnapshot	LiveRoom
id PK	id PK
account FK → SocialAccount	host FK → User criador
date Date data da foto	room_code CharField código curto único
followers Integer	title CharField
following Integer	room_type CharField live/meeting/webinar
posts_count Integer	status CharField scheduled/live/ended
reach Integer peessoas únicas	max_participants Integer
impressions Integer visualizações totais	is_private Boolean
profile_views Integer	password CharField sala privada
website_clicks Integer	recording_enabled Boolean
avg_likes_7d Float média 7 dias	recording_url URLField Supabase Storage
avg_comments_7d Float	agora_channel CharField canal no Agora.io
created_at DateTime auto	scheduled_at DateTime
	started_at DateTime
	ended_at DateTime
	created_at DateTime auto

■ PARTE 4 — SISTEMA DE USUÁRIOS COMPLETO

O Clerk cuida do registro, login e OAuth. O Django cuida dos dados de negócio (perfil, plano, equipe, permissões). O middleware sincroniza os dois automaticamente.

■ Fluxo completo de autenticação

■ Fluxo de autenticação

```
# Fluxo de autenticação — passo a passo:
#
# 1. Usuário acessa /login → Clerk exibe formulário (email/senha ou Google/GitHub)
# 2. Clerk autentica e retorna JWT assinado para o browser
# 3. Frontend envia JWT em TODAS as requisições:
# Header: Authorization: Bearer eyJhbGci...
# 4. ClerkAuthMiddleware do Django:
# a. Extrai o token do header Authorization
# b. Busca chaves públicas do Clerk (JWKS endpoint)
# c. Verifica a assinatura do JWT com as chaves
# d. Extrai user_id (campo 'sub') do payload
# e. Busca ou cria o User Django com aquele clerk_user_id
# f. Define request.user → todas as views funcionam normalmente
#
# 5. Views Django funcionam como sempre:
# @login_required → funciona (verifica request.user.is_authenticated)
# request.user.profile → dados do perfil
# request.user.social_accounts.all() → contas conectadas
# request.user.profile.is_plan_active → plano válido?
```

■ Middleware + settings

■ MIDDLEWARE

```
# config/settings/base.py — MIDDLEWARE (ordem importa!)
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'whitenoise.middleware.WhiteNoiseMiddleware',
    'apps.accounts.clerk_auth.ClerkAuthMiddleware', # ← Clerk JWT
    'django.contrib.sessions.middleware.SessionMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

# CORS — permitir o frontend chamar a API
CORS_ALLOWED_ORIGINS = ['https://seudominio.com', 'http://localhost:3000']
CORS_ALLOW_CREDENTIALS = True
```

■ Onboarding — primeiro login

■ apps/accounts/views.py — onboarding

```
# apps/accounts/views.py
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect
```

```

from django import forms
from .models import UserProfile

class OnboardingForm(forms.ModelForm):
    first_name = forms.CharField(max_length=50, label='Seu nome')
    last_name = forms.CharField(max_length=50, label='Sobrenome')
    class Meta:
        model = UserProfile
        fields = ['bio', 'timezone', 'language', 'phone', 'website']
    @login_required
    def onboarding(request):
        profile = request.user.profile
        if profile.onboarding_done:
            return redirect('dashboard')
        if request.method == 'POST':
            form = OnboardingForm(request.POST, instance=profile)
            if form.is_valid():
                request.user.first_name = form.cleaned_data['first_name']
                request.user.last_name = form.cleaned_data['last_name']
                request.user.save(update_fields=['first_name', 'last_name'])
            profile = form.save(commit=False)
            profile.onboarding_done = True
            profile.save()
            # Rastrear evento no PostHog
            from apps.analytics.posthog_client import track
            track(request.user.id, 'onboarding_completed')
            return redirect('connect_social') # próximo passo
        else:
            form = OnboardingForm(instance=profile)
        return render(request, 'accounts/onboarding.html', {'form': form})

```

■ RBAC — Permissões por papel

■ apps/accounts/permissions.py — RBAC

```

# apps/accounts/permissions.py
from functools import wraps
from django.shortcuts import redirect
from django.contrib import messages
from .models import TeamMembership

ROLE_PERMISSIONS = {
    'owner': ['post.*', 'account.*', 'team.*', 'analytics.view', 'billing.*'],
    'admin': ['post.*', 'account.connect', 'team.invite', 'analytics.view'],
    'editor': ['post.create', 'post.edit', 'post.publish', 'analytics.view'],
    'viewer': ['analytics.view'],
}

def has_permission(user, team, perm: str) -> bool:
    try:
        m = TeamMembership.objects.get(user=user, team=team, invite_accepted=True)
        allowed = ROLE_PERMISSIONS.get(m.role, [])
        # Suporte a wildcard: 'post.*' cobre 'post.create', 'post.edit', etc.
        for a in allowed:
            if a == perm: return True
            if a.endswith('.*') and perm.startswith(a[:-1]): return True
        return False
    except TeamMembership.DoesNotExist:
        return False

```

```
def require_permission(perm: str):
def decorator(view_func):
@wraps(view_func)
def wrapper(request, *args, **kwargs):
tid = kwargs.get('team_id') or request.session.get('active_team')
if tid:
from apps.accounts.models import Team
try:
if not has_permission(request.user, Team.objects.get(id=tid), perm):
messages.error(request, 'Sem permissão.')
return redirect('dashboard')
except Team.DoesNotExist:
return redirect('dashboard')
return view_func(request, *args, **kwargs)
return wrapper
return decorator

# Uso:
# @login_required
# @require_permission('post.delete')
# def delete_post(request, post_id): ...
```

■ Serializers e API REST (DRF)

■ Serializers + ViewSets

```
# apps/accounts/api/serializers.py
from rest_framework import serializers
from django.contrib.auth.models import User
from apps.accounts.models import UserProfile, Plan, Team, TeamMembership

class PlanSerializer(serializers.ModelSerializer):
class Meta:
model = Plan
fields = ['id', 'name', 'slug', 'price_monthly', 'has_ai_features',
'has_analytics', 'has_live_streaming', 'max_social_accounts']

class ProfileSerializer(serializers.ModelSerializer):
plan = PlanSerializer(read_only=True)
avatar_url = serializers.SerializerMethodField()
is_plan_active = serializers.ReadOnlyField()
class Meta:
model = UserProfile
fields = ['bio', 'avatar_url', 'phone', 'website', 'timezone',
'language', 'plan', 'is_plan_active', 'onboarding_done']
def get_avatar_url(self, obj):
if not obj.avatar: return None
req = self.context.get('request')
return req.build_absolute_uri(obj.avatar.url) if req else obj.avatar.url

class UserSerializer(serializers.ModelSerializer):
profile = ProfileSerializer(read_only=True)
full_name = serializers.SerializerMethodField()
class Meta:
model = User
fields = ['id', 'email', 'first_name', 'last_name', 'full_name',
'date_joined', 'last_login', 'profile']
def get_full_name(self, obj):
return obj.profile.full_name if hasattr(obj, 'profile') else ''

# apps/accounts/api/views.py
```

```
from rest_framework import viewsets, permissions
from rest_framework.decorators import action
from rest_framework.response import Response

class UserViewSet(viewsets.ModelViewSet):
    serializer_class = UserSerializer
    permission_classes = [permissions.IsAuthenticated]
    def get_queryset(self):
        return User.objects.filter(id=self.request.user.id)
    @action(detail=False, methods=['get', 'patch'])
    def me(self, request):
        if request.method == 'GET':
            return Response(self.get_serializer(request.user,
            context={'request':request}).data)
        for f in ['bio', 'phone', 'website', 'timezone', 'language']:
            if f in request.data:
                setattr(request.user.profile, f, request.data[f])
            request.user.profile.save()
        return Response({'status': 'ok'})
```

■ PARTE 5 — APIS SOCIAIS, AGENDAMENTO E IA

■ Serviço base para todas as redes

■ apps/social/services/base.py

```
# apps/social/services/base.py — interface comum para todas as redes
from abc import ABC, abstractmethod
from apps.social.models import SocialAccount

class BaseSocialService(ABC):
    def __init__(self, account: SocialAccount):
        self.account = account

    @abstractmethod
    def post_text(self, text: str) -> dict: ...

    @abstractmethod
    def post_with_media(self, text: str, media_paths: list) -> dict: ...

    @abstractmethod
    def get_metrics(self) -> dict: ...

    @abstractmethod
    def refresh_token(self) -> bool: ...

    def _token_header(self) -> dict:
        return {'Authorization': f'Bearer {self.account.access_token}'}
```

■ Twitter/X — Tweepy

■ apps/social/services/twitter.py

```
poetry add tweepy

# apps/social/services/twitter.py
import tweepy
from django.conf import settings
from .base import BaseSocialService

class TwitterService(BaseSocialService):
    def __init__(self, account):
        super().__init__(account)
        self.client = tweepy.Client(
            consumer_key=settings.TWITTER_API_KEY,
            consumer_secret=settings.TWITTER_API_SECRET,
            access_token=account.access_token,
            access_token_secret=account.refresh_token,
        )

    def post_text(self, text: str) -> dict:
        try:
            r = self.client.create_tweet(text=text[:280])
            return {'success': True, 'post_id': r.data['id'],
                'url': f'https://twitter.com/i/web/status/{r.data["id"]}' }
        except tweepy.TweepyException as e:
            return {'success': False, 'error': str(e)}

    def post_with_media(self, text, media_paths): ...

    def get_metrics(self): ...

    def refresh_token(self): ...
```

■ Celery — Configuração e Tasks de Publicação

■ config/celery.py

```
# config/celery.py
import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings.development')
app = Celery('socialhub')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks() # encontra tasks.py em todos os apps
```

■ apps/scheduler/tasks.py — publicação

```
# apps/scheduler/tasks.py
from celery import shared_task
from django.utils import timezone
from .models import ScheduledPost, PublicationResult
from apps.social.services.twitter import TwitterService

@shared_task(bind=True, max_retries=3)
def publish_scheduled_posts(self):
    """Roda a cada 60s - publica posts cujo horário chegou"""
    now = timezone.now()
    posts = ScheduledPost.objects.filter(
        status='scheduled',
        scheduled_at__lte=now,
    ).prefetch_related('accounts')
    for post in posts:
        post.status = 'publishing'
        post.save(update_fields=['status'])
        for account in post.accounts.filter(is_active=True):
            result = PublicationResult(post=post, account=account)
            try:
                # Seleciona o serviço correto pela plataforma
                service = get_service(account)
                text = post.full_text_for(account.platform)
                res = service.post_text(text)
                if res['success']:
                    result.status = 'success'
                    result.platform_post_id = res['post_id']
                    result.platform_post_url = res.get('url', '')
                else:
                    result.status = 'failed'
                    result.error_message = res['error']
            except Exception as exc:
                result.status = 'failed'
                result.error_message = str(exc)
            # Celery vai tentar novamente (max 3x)
            raise self.retry(exc=exc, countdown=300) # 5 min
            result.save()

        # Marcar o post como publicado ou falhou
        all_results = post.results.all()
        if all_results.filter(status='success').exists():
            post.status = 'published'
            post.published_at = timezone.now()
        else:
            post.status = 'failed'
        post.save(update_fields=['status', 'published_at'])
```

```
def get_service(account):
    """Retorna o serviço correto para a plataforma"""
    from apps.social.services.twitter import TwitterService
    # from apps.social.services.instagram import InstagramService
    services = {'twitter': TwitterService}
    cls = services.get(account.platform)
    if not cls: raise ValueError(f'Plataforma {account.platform} não suportada')
    return cls(account)
```

■ Celery Beat — agendamento

```
# config/settings/base.py — agendar tasks
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    # Verificar posts para publicar a cada 1 minuto
    'publicar-posts': {
        'task': 'apps.scheduler.tasks.publish_scheduled_posts',
        'schedule': 60.0,
    },
    # Coletar métricas das contas a cada 6 horas
    'coletar-metricas': {
        'task': 'apps.analytics.tasks.collect_account_metrics',
        'schedule': crontab(minute=0, hour='*/6'),
    },
    # Coletar hashtags trending a cada hora
    'hashtags-trending': {
        'task': 'apps.ai_engine.tasks.update_trending_hashtags',
        'schedule': crontab(minute=0),
    },
}
```

■ IA — Geração de conteúdo com GPT-4o

■ apps/ai_engine/services.py

```
# apps/ai_engine/services.py
from openai import OpenAI
from django.conf import settings

client = OpenAI(api_key=settings.OPENAI_API_KEY)

PLATFORM_RULES = {
    'twitter': 'Máximo 280 caracteres. Direto ao ponto.',
    'instagram': 'Pode ser longo. Use emojis. CTA no final.',
    'linkedin': 'Tom profissional. Foco em valor e aprendizado.',
    'tiktok': 'Jovem, divertido, gírias leves. CTA para comentar.',
}

def generate_post(topic: str, platform: str, tone: str = 'casual') -> dict:
    prompt = (
        f'Crie um post para {platform} sobre: {topic}.\n'
        f'Tom: {tone}.\n'
        f'Regras: {PLATFORM_RULES.get(platform, "")}\n'
        'Responda APENAS com o texto do post, sem aspas.'
    )
    response = client.chat.completions.create(
        model='gpt-4o-mini', # mais barato — use gpt-4o para melhor qualidade
        messages=[
            {'role': 'system', 'content': 'Você é especialista em marketing digital.'},
            {'role': 'user', 'content': prompt}
        ]
    )
```

```
],
max_tokens=600, temperature=0.8,
)
content = response.choices[0].message.content
tokens = response.usage.total_tokens
return {'content': content, 'tokens': tokens,
'cost_usd': round(tokens * 0.00000015, 6)}

def suggest_hashtags(topic: str, platform: str, n: int = 10) -> list[str]:
response = client.chat.completions.create(
model='gpt-4o-mini',
messages=[{'role':'user', 'content':
f'Sugira {n} hashtags em PT e EN para um post sobre "{topic}" no {platform}.'
' Retorne APENAS as hashtags separadas por espaço. Ex: #python #dev'}],
max_tokens=100,
)
return response.choices[0].message.content.strip().split()
```

■ PARTE 6 — INBOX, AUTO-RESPOSTA E LIVE STREAMING

■ WebSocket — Inbox em tempo real

■ **django-channels** O Django padrão é 'request-response' — o usuário pede, o servidor responde. Para o inbox aparecer novas mensagens sem recarregar, usamos WebSocket via django-channels. O Redis (Upstash) serve como canal de comunicação entre workers.

■ WebSocket — Inbox em tempo real

```
# config/asgi.py
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
import apps.inbox.routing

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings.development')
application = ProtocolTypeRouter({
    'http': get_asgi_application(),
    'websocket': AuthMiddlewareStack(
        URLRouter(apps.inbox.routing.websocket_urlpatterns)
    ),
})

# apps/inbox/consumers.py
import json
from channels.generic.websocket import AsyncWebsocketConsumer

class InboxConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.group = f'inbox_{self.scope["user"].id}'
        await self.channel_layer.group_add(self.group, self.channel_name)
        await self.accept()

    async def disconnect(self, code):
        await self.channel_layer.group_discard(self.group, self.channel_name)

    async def new_message(self, event):
        await self.send(text_data=json.dumps({
            'type': 'new_message',
            'platform': event['platform'],
            'sender': event['sender'],
            'text': event['text'],
            'timestamp': event['timestamp'],
        }))

# apps/inbox/routing.py
from django.urls import re_path
from . import consumers

websocket_urlpatterns = [re_path(r'ws/inbox/$', consumers.InboxConsumer.as_asgi())]
```

■ Auto-resposta com IA

■ apps/inbox/services.py

```
# apps/inbox/services.py — processar mensagem recebida
from django.contrib.auth.models import User
from .models import AutoReplyRule, InboxMessage, InboxReply
```

```
from apps.ai_engine.services import generate_reply

def process_message(user: User, platform: str, sender: str,
message_text: str, message_obj: InboxMessage) -> str | None:
    """
    Processa uma mensagem recebida e envia resposta automática se aplicável.
    Retorna o texto da resposta, ou None se não respondeu.
    """

    from django.utils import timezone
    rules = AutoReplyRule.objects.filter(
        user=user, is_active=True,
        applies_to__in=[platform, 'both']
    )

    for rule in rules:
        # Verificar se está dentro do horário ativo
        hour = timezone.now().hour
        if not (rule.active_hours_start <= hour <= rule.active_hours_end):
            continue

        # Verificar limite de respostas
        today = timezone.now().date()
        replies_today = InboxReply.objects.filter(
            message__user=user,
            is_auto_reply=True,
            created_at__date=today
        ).count()
        if replies_today >= rule.max_per_day:
            continue

        # Verificar trigger
        triggered = False
        if rule.trigger_type == 'all':
            triggered = True
        elif rule.trigger_type == 'keyword':
            keywords = [k.strip().lower() for k in rule.trigger_keywords.split(',')]
            triggered = any(kw in message_text.lower() for kw in keywords)

        if not triggered:
            continue

        # Gerar resposta
        if rule.reply_mode == 'fixed':
            reply_text = rule.fixed_reply
        else:
            reply_text = generate_reply(
                user_message=message_text,
                context=rule.ai_instructions,
                tone=rule.ai_tone,
            )

        # Salvar resposta no banco
        InboxReply.objects.create(
            message=message_obj,
            content=reply_text,
            is_auto_reply=True,
            status='sent',
        )

        message_obj.is_replied = True
        message_obj.auto_replied = True
        message_obj.save(update_fields=['is_replied', 'auto_replied'])
        rule.reply_count += 1
```

```
rule.save(update_fields=['reply_count'])
return reply_text

return None # nenhuma regra se aplicou
```

■ Live Streaming com Agora.io

■ apps/live/ — Agora.io

```
poetry add agora-token-builder

# .env:
AGORA_APP_ID=xxxxxxxxxxxxxxxxxxxxxxxx
AGORA_APP_CERTIFICATE=xxxxxxxxxxxxxxxxxxxxxxxx

# apps/live/services.py
from agora_token_builder import RtcTokenBuilder, RtcRole
import time, uuid
from django.conf import settings

def generate_agora_token(channel: str, uid: int) -> str:
    expiry = int(time.time()) + 3600 # 1 hora
    return RtcTokenBuilder.buildTokenWithUid(
        settings.AGORA_APP_ID, settings.AGORA_APP_CERTIFICATE,
        channel, uid, RtcRole.PUBLISHER, expiry, expiry
    )

# apps/live/views.py
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse
from .models import LiveRoom
from .services import generate_agora_token

@login_required
def create_room(request):
    code = str(uuid.uuid4())[:8]
    room = LiveRoom.objects.create(
        host=request.user, room_code=code,
        title=request.POST.get('title', 'Minha Live'),
        room_type=request.POST.get('type', 'meeting')
    )
    token = generate_agora_token(code, request.user.id)
    return JsonResponse({'room_id': room.id, 'room_code': code, 'token': token,
        'app_id': settings.AGORA_APP_ID})

@login_required
def get_room_token(request, code: str):
    token = generate_agora_token(code, request.user.id)
    return JsonResponse({'token': token, 'app_id': settings.AGORA_APP_ID,
        'channel': code, 'uid': request.user.id})
```

PARTE 7 — PRODUÇÃO: DEPLOY, CI/CD, HTTPS E MONITORAMENTO

Esta parte cobre tudo necessário para colocar o SocialHub Pro no ar de forma segura, automatizada e monitorada. Siga os passos em ordem.

Checklist pré-deploy

✓	Item	Como verificar
■	DEBUG = False no settings de produção	settings/production.py
■	SECRET_KEY diferente do desenvolvimento	.env de produção no Vercel
■	DATABASE_URL apontando para Supabase	Vercel → Env Vars
■	ALLOWED_HOSTS com domínio real	settings/production.py
■	HTTPS configurado no Cloudflare	SSL/TLS → Full (strict)
■	Todas as chaves de API no Vercel Env Vars	Vercel → Settings → Env
■	requirements.txt gerado e atualizado	poetry export
■	Testes passando 100%	poetry run pytest
■	collectstatic rodou sem erros	manage.py collectstatic
■	Sentry DSN configurado	settings/production.py
■	Workers Celery no Railway	Railway → deploy ativo

Cloudflare — DNS + HTTPS + CDN (gratuito)

■ Cloudflare — passo a passo

```
# 1. cloudflare.com → Add a Site → inserir seudominio.com
# 2. Selecione plano Free
# 3. O Cloudflare mostra os nameservers dele:
# Ex: alice.ns.cloudflare.com e bob.ns.cloudflare.com
# 4. No Namecheap → Domain → Nameservers → Custom DNS
# Cole os 2 nameservers do Cloudflare
# (propagação leva até 48h, geralmente <1h)
#
# 5. No Cloudflare → DNS → Records → Adicionar registros do Vercel:
# Type: CNAME Name: @ Target: cname.vercel-dns.com
# Type: CNAME Name: www Target: cname.vercel-dns.com
#
# 6. Adicionar registros do Resend (e-mail):
# O Resend mostra os registros exatos ao adicionar o domínio
#
# 7. SSL/TLS → Encryption Mode → Full (strict)
# Ativa HTTPS automático – certificado gerenciado pelo Cloudflare
#
# 8. Regras de segurança opcionais (WAF – Free tier):
# Security → WAF → Managed Rules → ativar OWASP Core Rules
```

■ CI/CD — GitHub Actions

■ .github/workflows/test.yml

```
# .github/workflows/test.yml
# Roda testes em todo Pull Request (antes de mergear)
name: Tests
on:
  pull_request:
  branches: [main, develop]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_DB: socialhub_test
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: postgres
        options: >-
        --health-cmd pg_isready --health-interval 10s
        --health-timeout 5s --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      with: {python-version: '3.12'}
      - name: Install Poetry
        run: pip install poetry
      - name: Install deps
        run: poetry install
      - name: Run tests
        env:
          DATABASE_URL: postgres://postgres:postgres@localhost/socialhub_test
          DJANGO_ENV: development
          SECRET_KEY: test-secret-key-only-for-ci
          REDIS_URL: redis://localhost:6379
        run: poetry run pytest --tb=short -q
      - name: Check code style
        run: poetry run ruff check .
```

■ .github/workflows/deploy.yml

```
# .github/workflows/deploy.yml
# Deploy automático ao mergear no main (após testes passarem)
name: Deploy to Production
on:
  push:
  branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      with: {python-version: '3.12'}
      - name: Install deps e gerar requirements.txt
```

```

run: |
pip install poetry
poetry install
poetry export -f requirements.txt --output requirements.txt --without-hashes
- name: Deploy no Vercel
uses: amondnet/vercel-action@v25
with:
vercel-token: ${ secrets.VERCEL_TOKEN }
vercel-org-id: ${ secrets.VERCEL_ORG_ID }
vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
vercel-args: '--prod'
- name: Notificar Sentry do deploy
env:
SENTRY_AUTH_TOKEN: ${ secrets.SENTRY_AUTH_TOKEN }
SENTRY_ORG: seu-org
SENTRY_PROJECT: socialhub
run: |
pip install sentry-cli
sentry-cli releases new ${ github.sha }
sentry-cli releases finalize ${ github.sha }

# Secrets necessários no GitHub → Settings → Secrets:
# VERCEL_TOKEN → Vercel → Account → Settings → Tokens
# VERCEL_ORG_ID → vercel.json ou .vercel/project.json
# VERCEL_PROJECT_ID → idem
# SENTRY_AUTH_TOKEN → Sentry → Settings → API → Auth Tokens

```

■ Docker — Workers Celery em produção (Railway)

■ docker/Dockerfile.worker + docker-compose.prod.yml

```

# docker/Dockerfile.worker
FROM python:3.12-slim

# Dependências do sistema
RUN apt-get update && apt-get install -y \
gcc libpq-dev curl && rm -rf /var/lib/apt/lists/*
WORKDIR /app

# Instalar dependências Python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar código
COPY . .

# Variáveis
ENV DJANGO_ENV=production
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Sem CMD — definido pelo Railway por serviço

# docker/docker-compose.prod.yml
# Use apenas para testar localmente o ambiente de produção
version: '3.8'
services:
worker:
build:
context: ..
dockerfile: docker/Dockerfile.worker
command: celery -A config worker --loglevel=info --concurrency=2

```

```

env_file: ../.env
restart: unless-stopped

beat:
build:
context: ..
dockerfile: docker/Dockerfile.worker
command: celery -A config beat --loglevel=info
env_file: ../.env
restart: unless-stopped

flower:
build:
context: ..
dockerfile: docker/Dockerfile.worker
command: celery -A config flower --port=5555
ports: ['5555:5555']
env_file: ../.env

```

■ Migrações em produção (Supabase)

■ Migrações em produção

```

# NUNCA rode migrate manualmente em produção sem backup!
#
# Fluxo seguro para aplicar migrations:
#
# 1. Criar migration em dev e testar localmente
poetry run python manage.py makemigrations
poetry run python manage.py migrate # testa em dev
# 2. Commitar a migration junto com o código
git add apps/*/migrations/ && git commit -m "db: migration para X"
# 3. Em produção – o GitHub Actions pode rodar automaticamente:
# Adicione ao deploy.yml após o deploy do Vercel:
#
- name: Rodar migrações
run: |
poetry run python manage.py migrate --settings=config.settings.production
#
# 4. Ou via Vercel CLI (manual quando necessário):
vercel run --prod python manage.py migrate
# 5. Backup antes de migrations destrutivas:
# Supabase → Settings → Database → Backups → Create backup

```

■ Sentry + PostHog — Monitoramento

■ Monitoramento

```

# apps/analytics/posthog_client.py
import posthog
from django.conf import settings

posthog.project_api_key = settings.POSTHOG_API_KEY
posthog.host = 'https://app.posthog.com'
posthog.disabled = settings.DEBUG # não rastreia em dev

def track(user_id: int, event: str, props: dict = None):
    posthog.capture(str(user_id), event, properties=props or {})

# Exemplos de eventos importantes para rastrear:
# track(user.id, 'post_scheduled', {'platform': 'twitter', 'ai': True})

```

```
# track(user.id, 'post_published', {'platform': 'twitter'})
# track(user.id, 'post_failed', {'platform': 'twitter', 'error': 'rate_limit'})
# track(user.id, 'plan_upgraded', {'from': 'free', 'to': 'pro', 'value': 49.90})
# track(user.id, 'live_started', {'room_type': 'meeting'})
# track(user.id, 'auto_reply_sent', {'platform': 'instagram'})
# track(user.id, 'team_created', {'members': 3})

# Sentry — configurado em settings/production.py (já mostrado na Parte 1)
# Sentry captura automaticamente:
# - Todas as exceções não tratadas no Django
# - Erros nas tasks Celery
# - Performance de queries lentas
# - Rastreamento de usuário (sem PII)
```

■ Stripe — Pagamentos para planos pagos

■ apps/payments/views.py — Stripe

```
# apps/payments/views.py
import stripe

from django.conf import settings
from django.http import JsonResponse
from django.contrib.auth.decorators import login_required
from django.views.decorators.csrf import csrf_exempt
from apps.accounts.models import Plan

stripe.api_key = settings.STRIPE_SECRET_KEY

@login_required
def create_checkout(request, plan_slug: str):
    plan = Plan.objects.get(slug=plan_slug)
    session = stripe.checkout.Session.create(
        payment_method_types=['card'],
        mode='subscription',
        customer_email=request.user.email,
        line_items=[{'price': plan.stripe_price_id, 'quantity': 1}],
        success_url=f'{settings.SITE_URL}/plano/sucesso/?session={{CHECKOUT_SESSION_ID}}',
        cancel_url=f'{settings.SITE_URL}/planos/',
        metadata={'user_id': str(request.user.id), 'plan_slug': plan_slug},
    )
    return JsonResponse({'url': session.url})

@csrf_exempt
def stripe_webhook(request):
    """Configure em: Stripe → Webhooks → Add endpoint → /payments/webhook/"""
    try:
        event = stripe.Webhook.construct_event(
            request.body,
            request.META['HTTP_STRIPE_SIGNATURE'],
            settings.STRIPE_WEBHOOK_SECRET
        )
    except Exception:
        return JsonResponse({'error': 'invalid'}, status=400)

    if event['type'] == 'checkout.session.completed':
        session = event['data']['object']
        user_id = int(session['metadata']['user_id'])
        plan_slug = session['metadata']['plan_slug']
        from django.contrib.auth.models import User
        from django.utils import timezone
        from datetime import timedelta
```

```
user = User.objects.get(id=user_id)
plan = Plan.objects.get(slug=plan_slug)
user.profile.plan = plan
user.profile.plan_expires_at = timezone.now() + timedelta(days=30)
user.profile.stripe_customer_id = session.get('customer', '')
user.profile.save()

elif event['type'] == 'customer.subscription.deleted':
    # Plano cancelado → voltar para free
    customer_id = event['data']['object']['customer']
    profile = UserProfile.objects.get(stripe_customer_id=customer_id)
    profile.plan = Plan.objects.get(slug='free')
    profile.plan_expires_at = None
    profile.save()

return JsonResponse({'status': 'ok'})
```

■ APÊNDICE — REFERÊNCIA RÁPIDA

■ Todos os models — resumo

App	Model	Propósito	Relações chave
accounts	Plan	Planos de assinatura e limites	1:N UserProfile
accounts	UserProfile	Extensão do User: plano, Clerk ID, avatar	1:1 User, FK Plan
accounts	Team	Equipes com múltiplos membros e papéis	FK User, M2M via TeamMembership
accounts	TeamMembership	Papel de cada membro na equipe	FK Team + User
accounts	Notification	Notificações in-app	FK User
accounts	AuditLog	Histórico de todas as ações	FK User
social	SocialAccount	Conta OAuth2 conectada de uma rede social	FK User, FK Team
scheduler	ScheduledPost	Post agendado com conteúdo por plataforma	FK User, M2M SocialAccount
scheduler	PostMedia	Imagens/vídeos de um post	FK ScheduledPost
scheduler	PublicationResult	Resultado da publicação por rede social	FK ScheduledPost + SocialAccount
inbox	InboxMessage	DM/comentário recebido de qualquer rede	FK User, FK SocialAccount
inbox	InboxReply	Resposta enviada a uma mensagem	FK InboxMessage, FK User
inbox	AutoReplyRule	Regra de resposta automática por IA/keyword	FK User
analytics	AccountSnapshot	Foto diária de seguidores e alcance	FK SocialAccount
analytics	PostMetrics	Métricas de engajamento por post	FK SocialAccount
live	LiveRoom	Sala de live streaming ou reunião	FK User (host)
live	LiveParticipant	Participante de uma sala ao vivo	FK LiveRoom, FK User
payments	Subscription	Assinatura ativa do Stripe	FK User, FK Plan
ai_engine	AIGenerationLog	Histórico de uso da IA com custo em tokens	FK User
ai_engine	HashtagCache	Cache de hashtags trending por região	—

■ Comandos do dia a dia

Referência de comandos

[illegible]

[illegible]

■ Variáveis de ambiente — template .env.example

- .env.example completo

```
# .env.example - commitar no Git (sem valores reais!)
# Cada dev cria seu .env copiando este arquivo

# Django
DJANGO_ENV=development
SECRET_KEY=django-insecure-mude-esta-chave-em-producao
SITE_URL=http://localhost:8000
ALLOWED_HOST=seudominio.com

# Supabase - Banco de dados
DATABASE_URL=postgresql://postgres:SENHA@db.SEUID.supabase.co:5432/postgres
SUPABASE_S3_URL=https://SEUID.supabase.co/storage/v1/s3
SUPABASE_BUCKET_NAME=socialhub-media
SUPABASE_ACCESS_KEY=
SUPABASE_SECRET_KEY=

# Redis - Upstash
REDIS_URL=redis://:SENHA@global-xxx.upstash.io:6379

# Clerk - Autenticação
CLERK_SECRET_KEY=sk_test_
CLERK_JWKS_URL=https://SEUAPP.clerk.accounts.dev/.well-known/jwks.json

# Resend - E-mails
RESEND_API_KEY=re_
DEFAULT_FROM_EMAIL=noreply@seudominio.com

# OpenAI - IA
OPENAI_API_KEY=sk-

# Stripe - Pagamentos
STRIPE_SECRET_KEY=sk_live_
STRIPE_WEBHOOK_SECRET=whsec_

# PostHog - Analytics
POSTHOG_API_KEY=phc
```

```
# Sentry — Erros
SENTRY_DSN=https://xxx@xxx.ingest.sentry.io/xxx

# Agora.io — Live streaming
AGORA_APP_ID=
AGORA_APP_CERTIFICATE=

# APIs das redes sociais
TWITTER_API_KEY=
TWITTER_API_SECRET=
INSTAGRAM_APP_ID=
INSTAGRAM_APP_SECRET=
LINKEDIN_CLIENT_ID=
LINKEDIN_CLIENT_SECRET=
TIKTOK_CLIENT_KEY=
TIKTOK_CLIENT_SECRET=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
```

■ Ordem correta para iniciar o projeto:

1. **Criar contas** → GitHub, Supabase, Clerk, Upstash, Resend, Vercel, PostHog, Sentry
2. **Setup local** → pyenv + Python 3.12 + Poetry + criar projeto Django
3. **Preencher .env** → DATABASE_URL, REDIS_URL, CLERK_SECRET_KEY, etc.
4. **Criar models** → na ordem da tabela desta parte (evita erros de FK)
5. **migrate + createsuperuser** → testar acesso ao /admin
6. **Conectar 1 rede social** → publicar post de teste via código
7. **Deploy no Vercel** → vercel --prod + configurar Cloudflare + Namecheap
8. **Workers no Railway** → Celery worker + beat rodando 24/7